

DVWA SQL Injection Report

Objective:

Install DVWA in Kali Linux, perform SQL Injection to extract usernames and password hashes, and demonstrate prevention using Prepared Statements.

1. Introduction:

Damn Vulnerable Web Application (DVWA) is a deliberately vulnerable web application used for learning web security concepts. In this task, DVWA was used to demonstrate SQL Injection and understand how prepared statements mitigate the vulnerability.

2. DVWA Setup:

DVWA was installed and configured on Kali Linux. The application was accessed through <http://localhost/DVWA/> and the security level was configured to Low for testing.

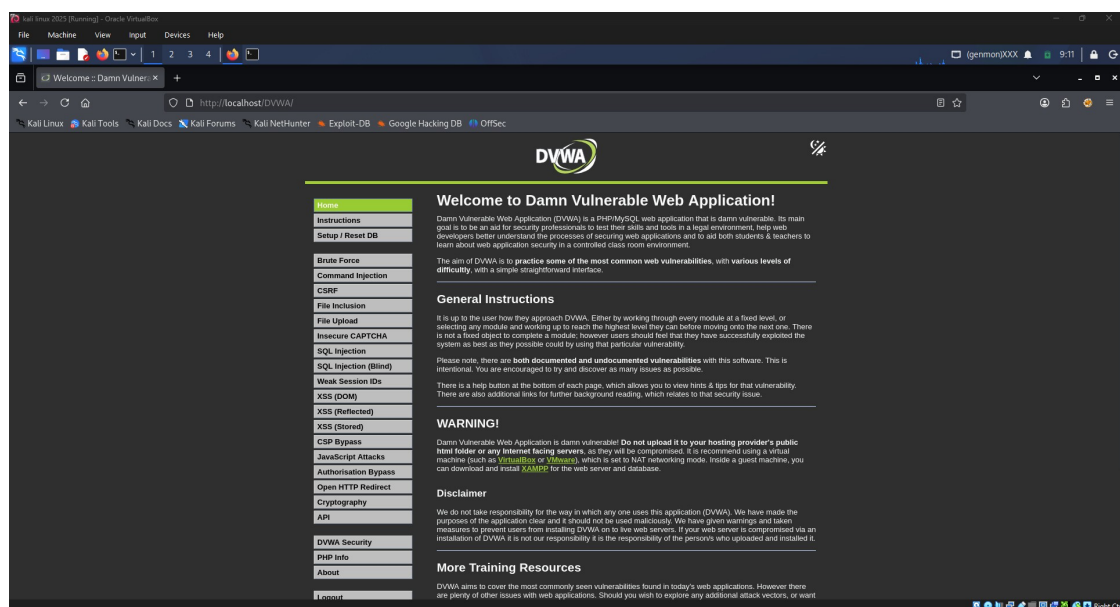


Figure 1: DVWA Home Page

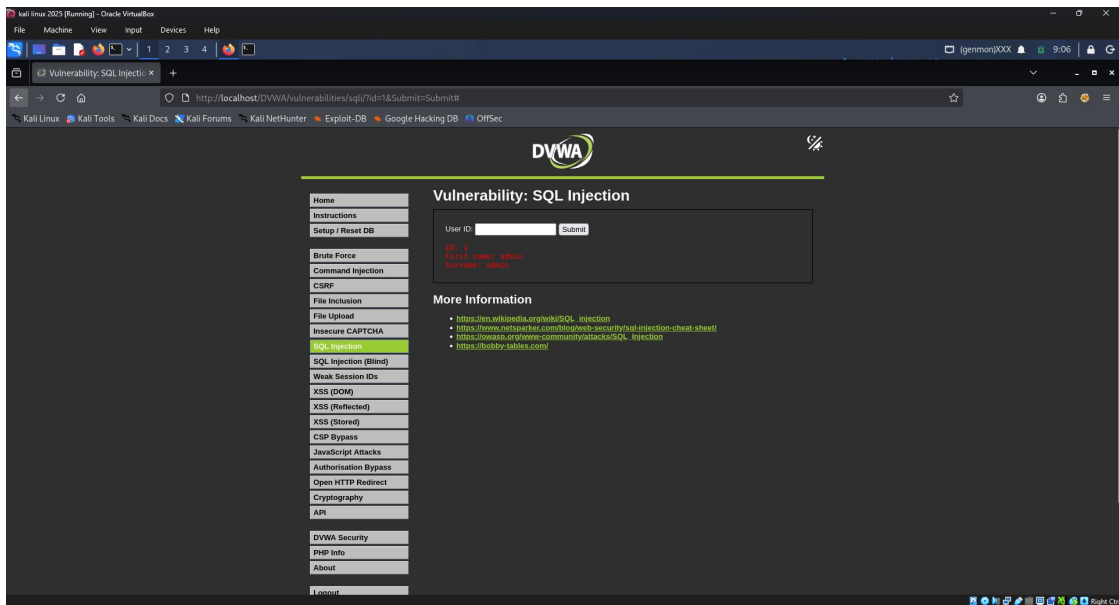


Figure 2: Normal Query Using User ID 1

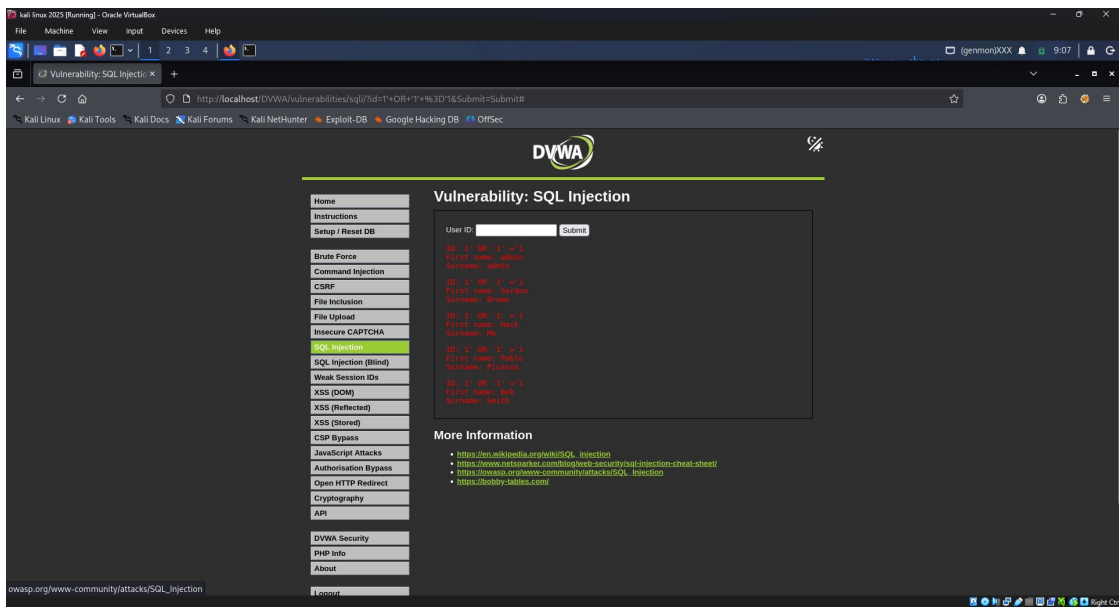


Figure 3: SQL Injection Payload 1' OR '1'='1

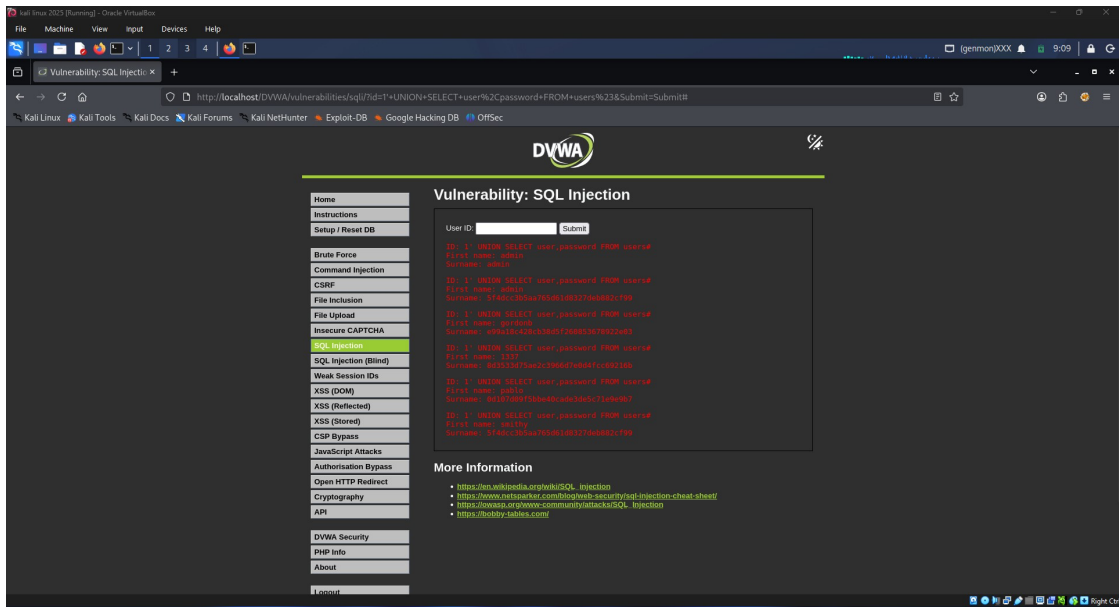


Figure 4: Username and Password Hash Extraction using UNION SELECT

3. SQL Injection Demonstration:

A normal query was first performed using User ID 1, which returned only the record belonging to the admin user. Next, the payload 1' OR '1'='1 was entered. This modified the SQL query logic and caused the database to return multiple records.

4. Extraction of Usernames and Password Hashes:

The following payload was used:

```
1' UNION SELECT user,password FROM users#
```

This UNION query combined results from the users table and displayed usernames along with their password hashes, demonstrating the impact of SQL Injection.

5. Prevention Using Prepared Statements:

Vulnerable Code:

```
SELECT first_name, last_name FROM users WHERE user_id = '$id';
```

In the vulnerable implementation, user input is directly inserted into the SQL query. An attacker can modify the query structure using malicious input.

Secure Code Using Prepared Statements:

```
$stmt = $pdo->prepare("SELECT first_name, last_name FROM users WHERE user_id = :id");  
$stmt->bindParam(':id', $id);  
$stmt->execute();
```

Prepared statements separate SQL commands from user input. The database first compiles the query structure and then binds user data. As a result, malicious input is treated as data instead of executable SQL, preventing SQL Injection attacks.

6. Conclusion:

The SQL Injection vulnerability was successfully demonstrated in DVWA. Usernames and password hashes were extracted using malicious SQL input. The vulnerability can be effectively mitigated through the use of Prepared Statements and parameterized queries.